

Skills Profile — Web & API Security

Name: Maturi Jaswanth Sai Madhu Mohan

Title: Web & API Penetration Tester — OSINT & Automated Scanning Specialist

Contact: collegegamerff@gmail.com | +91 82470 60862

LinkedIn: <https://www.linkedin.com/in/maturi-jaswanth-sai-madhu-mohan>

GitHub:- <https://github.com/jaswanthsai1>



Professional summary

Experienced penetration tester focused on web and API security, with practical experience in OSINT, automated scanning, and introductory threat-intelligence workflows. Skilled at combining manual techniques and automation to discover authentication/authorization flaws, SSRF, injection issues, business-logic problems, and misconfigurations. Produces clear, actionable reports and reproducible PoCs for development and security teams.

Core skills & proficiency (at-a-glance)

- Web Application Penetration Testing — Advanced
- API Penetration Testing (REST, GraphQL, SOAP) — Advanced
- OSINT (Open-Source Intelligence) — Intermediate–Advanced
- Threat Intelligence (basics & operationalization) — Beginner–Intermediate
- Automated Scanning & CI Integration — Intermediate
- Vulnerability Reporting & Proof-of-Concept (PoC) — Advanced

API Penetration Testing

What it is

Focused testing of APIs (REST, GraphQL, SOAP) to find flaws in authentication, authorization, business logic, schema design, input validation, and data exposure. API testing uncovers issues that UI-based tests often miss because APIs frequently expose functionality and data directly.

Typical testing steps

- Inventory endpoints — Collect documented and undocumented endpoints, mobile-app endpoints, and internal/admin APIs (use OpenAPI/Swagger, mobile app traffic captures).
- Authentication & session tests — Validate JWT/OAuth handling, token expiry, refresh flows, scope enforcement, replayability and token revocation.
- Authorization checks — Test for broken object-level authorization (BOLA), horizontal/vertical privilege escalation, and access control bypasses.
- Input fuzzing & validation — Fuzz parameters, content-types, boundary values, and malformed JSON/GraphQL queries.
- Business logic testing — Check rate limits, monetary flows, race conditions, and inconsistent state transitions.
- Schema & data exposure — Verify endpoints do not leak PII/excessive fields, debug info, or admin-only fields.
- Error handling & injection — Look for SQLi, NoSQLi, command injection, header injection, and overly descriptive errors.
- Performance & abuse — Test for abuse like mass data extraction, pagination abuse, and resource exhaustion.

Common tools

Postman, Insomnia (manual exploration + automation) Burp Suite (proxy, Intruder, Repeater) + useful extensions (AuthMatrix, JSON Beautifier, Logger++) OWASP ZAP, ffuf, dirsearch (discovery) GraphQL tools: GraphiQL, InQL, graphql-fuzzer soapUI for SOAP; JWT/OAuth debugging tools Nuclei, custom Python/JS scripts, and scripts/enumerators for object ID discovery

Example findings (typical)

- BOLA: changing an object ID allows access to another user's resource.
- Excessive data exposure: endpoints returning sensitive fields that should be omitted.
- Weak token validation: expired or tampered tokens accepted, or token scope ignored.
- Rate limiting bypass: endpoints allow large-scale scraping of data.

Mitigations & best practices

Server-side authorization for each object; never trust client-side checks. Use least-privilege tokens, validate scopes/claims, and short token lifetimes. Apply strict input validation and output filtering; use DTOs to limit returned fields. Rate-limit per user/IP and implement anomaly detection for bulk access patterns. Monitor and log anomalous API usage for faster detection.

Web Application Penetration Testing

What it is

Comprehensive assessment of web applications to find vulnerabilities (OWASP Top 10 and beyond): XSS, SQL injection, CSRF, authentication/session weaknesses, insecure deserialization, RCE, and business-logic flaws.

Typical testing steps

- Reconnaissance: Passive + active discovery (subdomains, endpoints, certificate transparency).
- Mapping: Understand functionality, workflows, and user roles.
- Automated scanning: Identify obvious issues with safe configuration.
- Manual testing: Deep investigation for logic flaws and chained attack paths.
- Exploitation & PoC: Create safe, reproducible PoCs demonstrating impact.
- Reporting & remediation verification: Provide prioritized fixes and retest.

Common tools

Burp Suite (proxy, scanner, intruder, repeater), OWASP ZAP Nmap, Nikto, sqlmap, ffuf/dirsearch for discovery Browser devtools, headless browsers, custom scripts (Python/JS)

Mitigations & best practices

Secure coding practices, strict input validation, and output encoding. Implement CSP, secure cookie flags, and robust session management. Harden server/configurations and patch dependencies regularly. Conduct threat modeling and security reviews before production rollout.

Server-Side Request Forgery (SSRF)

Overview & impact

SSRF allows an attacker to coerce a server to make requests to internal/external systems. Impacts include access to internal-only services, cloud metadata APIs (credential exposure), internal port scanning, data exfiltration, and pivoting into internal networks.

Detection & testing approach

Identify features that fetch user-supplied URLs (image fetch, URL preview, importers). Test alternate host notations (IPv4 integer/hex/octal), URL-encoding, redirects, credentials-in-host and nested/encoded redirects. Use OAST techniques (Burp Collaborator, Interact.sh, custom callback servers) for blind SSRF.

Common bypass patterns

IP encoding (e.g., 127.0.0.1 encoded as integer), nested redirects, host header manipulation, DNS rebinding.

Tools & techniques

Burp Suite + Collaborator, Interact.sh, ssrfmap, curl/httpie, custom callback servers and DNS OOB services.

Mitigations

Whitelist allowed destination domains; perform DNS resolution + normalization before validation. Block internal IP ranges in application logic and at network egress. Avoid fetching arbitrary user-provided hosts for sensitive operations; use proxying with allowlists and least privilege.

OSINT (Open-Source Intelligence)

What it is

Collecting and analyzing publicly available information to map an organization's digital footprint, exposed assets, personnel, and likely attack vectors.

Typical workflow

- Asset discovery: Subdomains and cert logs (crt.sh), Shodan/Censys for exposed services.
- Public code & data search: GitHub, paste sites, misconfigured cloud storage (S3/GCS).
- People & social profiling: LinkedIn, social networks for spear-phishing surface.
- Cross-correlation: Prioritize assets by business criticality and exploitability.

Common tools

Amass, subfinder, assetfinder, crt.sh, Shodan, Censys, GitLeaks, theHarvester, Maltego, SpiderFoot

Ethics & scope

Always operate within legal boundaries and the defined rules of engagement.

Threat Intelligence (introductory / operational)

What it is

Collecting, enriching, and operationalizing indicators (IOCs) and attacker TTPs to aid detection, response, and decision-making for security operations.

Typical activities

Ingest threat feeds, normalize IOCs (IPs, domains, hashes). Enrich indicators (WHOIS, passive DNS, ASN lookups). Map activity to MITRE ATT&CK and produce prioritized, contextual intelligence. Create detection content and operational blocklists for SOC teams.

Common tools & feeds

MISP, OpenCTI, VirusTotal, passive DNS providers, abuseIPDB, Recorded Future (commercial)

Practical outputs

Detection rules (Sigma), verified IOCs, intelligence reports with recommended mitigations.

Automated Scanning & CI Integration

What it is

Using automation to accelerate discovery and shift-left security checks into development workflows while minimizing false positives through validation and triage.

Typical approach

- Configure & scope: Exclude destructive endpoints; set safe scan policies.
- Run templated scans: Nuclei, Nikto, Trivy for container and infrastructure checks.
- Triage & validate: Manually verify high-confidence findings.
- CI/CD integration: Lightweight PR scans and scheduled full-scans on builds.

Common tools

Nuclei + templates, OWASP ZAP (CI), Burp Scanner (automation), Trivy (containers), Dependabot, Snyk (dependency checks)

Best practices

Fail builds only on verified critical issues; create developer tickets for actionable remediation. Keep scanner noise low by filtering known false positives.

Reporting & Proof-of-Concept (PoC)

What my reports include

- Executive summary: High-level impact and business risk.
- Affected components: Vulnerable URLs/endpoints and environment details.
- Reproduction steps: Step-by-step instructions, sample requests/responses, and screenshots.
- Proof-of-Concept: Minimal, safe PoC demonstrating impact.
- Impact & remediation: Concrete fixes and code-level suggestions.
- Severity & metrics: CVSS v3.1 score and contextual business impact.
- Verification steps: How to re-test after fixes.

Reporting style: Clear, prioritized, actionable — suitable for both technical teams and management.

Example project / achievement entries

Fraud Detection in Transaction and Reporting System

- Developed a system to identify and report fraudulent transactions using
- behavioral analysis and anomaly detection.
- Incorporated basic machine learning and security checks to enhance fraud
- prevention.
- Created an automated bug Bounty tool kit that helps to scan and detect bug in any website with single command.
- Detected SSRF in an import/URL-preview feature via OAST and helped implement allowlists and DNS-normalization to remediate.

Certifications & continuous learning

- 1.CEH — Ongoing.
- 2.Completed various cyber security internships and hackathons

Ongoing hands-on practice: OWASP Juice Shop, PortSwigger Academy, CTF participation. Recommended next steps: OSCP / eJPT (for offensive depth), API-specific secure coding courses, and threat-hunting workshops.

How I work (process & collaboration)

1. Define scope & ROE: Targets, testing windows, destructive exclusions.
2. Recon & enumeration: Automated + manual discovery to prioritize targets.
3. Manual deep testing: Focus on auth/authorization, SSRF, logic flaws, and critical flows.
4. Validation & PoC: Reproduce with minimal impact and collect evidence.
5. Deliver report & Remediation guidance: Prioritized findings, clear fixes, verification steps.
6. Retest & close loop: Confirm fixes and provide final validation.

☐ Color theme for emphasis: #C49F8C

Made with **GAMMA**